

The Basics of Neural Networks

Greg Strabel

October 13, 2023

1 Assumptions

A neural network is a directed graph $G = (V, A)$ composed of nodes, V , and arrows, A . Assume that the nodes of the network have been indexed by a subscript $k \in \{1, \dots, K\}$ and that the output of the k^{th} neuron is $o_k = \sigma_k(\psi_k)$ where $\sigma_k : \mathbb{R} \rightarrow \mathbb{R}$ is an activation function and

$$\psi_k = \sum_{l:(l,k) \in A} w_{kl} o_l + \beta_k \quad (1)$$

where $w_{kl} \in \mathbb{R}$ is the weight applied to the input $o_l \in \mathbb{R}$ and $\beta_k \in \mathbb{R}$ is a bias term. If the k^{th} neuron is in the first layer, then the o_l are just the inputs to the neural network.

2 Back-propagation

Neural networks are typically trained using an algorithm called back-propagation. Given a loss function $L : \mathbf{Z} \times \mathbf{Y} \rightarrow \mathbb{R}$, where $\mathbf{Z}$ is the co-domain of the network, the chain rule of differentiation implies:

$$\frac{\partial L}{\partial w_{kj}} = \frac{\partial L}{\partial \psi_k} \frac{\partial \psi_k}{\partial w_{kj}} = \lambda_k o_j \quad (2)$$

and

$$\frac{\partial L}{\partial \beta_k} = \frac{\partial L}{\partial \psi_k} = \lambda_k \quad (3)$$

where

$$\lambda_k = \frac{\partial L}{\partial \psi_k} = \sum_{l:(k,l) \in A} \frac{\partial L}{\partial \psi_l} \frac{\partial \psi_l}{\partial o_k} \frac{\partial o_k}{\partial \psi_k} = \sum_{l:(k,l) \in A} \lambda_l w_{lk} \sigma'_k(\psi_k) \quad (4)$$

This is the back-propagation equation which we can use to work backwards through the network to update parameters. Suppose we have a set of data $\{(x_i, y_i)\}_{i=1 \dots m}$. Starting at terminal node l , we have:

$$\lambda_{li} = \frac{\partial L_i}{\partial \psi_{li}} = \frac{\partial L_i}{\partial z_{li}} \sigma'_l(\psi_{li}) \quad (5)$$

which suggests the updates:

$$w_{kl} \leftarrow w_{kl} - \eta \sum_{i=1}^m \frac{\partial L_i}{\partial z_{li}} \sigma'_l(\psi_{li}) o_{li} = w_{kl} - \eta \sum_{i=1}^m \lambda_{li} o_{li} \quad (6)$$

and

$$\beta_l \leftarrow \beta_l - \eta \sum_{i=1}^m \frac{\partial L_i}{\partial z_{li}} \sigma'_l(\psi_{li}) = \beta_l - \eta \sum_{i=1}^m \lambda_{li} \quad (7)$$

where η is the learning rate. Proceed recursively with the updates:

$$w_{kj} \leftarrow w_{kj} - \eta \sum_{i=1}^m \frac{\partial L_i}{\partial w_{kj}} = w_{kj} - \eta \sum_{i=1}^m \lambda_{ki} o_{ji} \quad (8)$$

and

$$\beta_k \leftarrow \beta_k - \eta \sum_{i=1}^m \frac{\partial L_i}{\partial \beta_k} = \beta_k - \eta \sum_{i=1}^m \lambda_{ki} \quad (9)$$

using equation (4) to produce the summands.

3 Dropout

Dropout is a method for preventing overfitting. For each mini-batch during training, rather than using the linear predictor

$$\psi_k = \sum_{l:(l,k) \in A} w_{kl} o_l + \beta_k \quad (10)$$

in the k^{th} neuron we instead use

$$\psi_k = \sum_{l:(l,k) \in A} w_{kl} r_{kl} o_l + \beta_k \quad (11)$$

where $r_{kl} \sim \text{Bernoulli}(p_{kl})$ and $p_{kl} \in [0, 1]$. This has the effect of changing the chain rule to:

$$\frac{\partial L}{\partial w_{kj}} = \frac{\partial L}{\partial \psi_k} \frac{\partial \psi_k}{\partial w_{kj}} = \lambda_k o_j r_{kj} \quad (12)$$

where now

$$\lambda_k = \frac{\partial L}{\partial \psi_k} = \sum_{l:(k,l) \in A} \frac{\partial L}{\partial \psi_l} \frac{\partial \psi_l}{\partial o_k} \frac{\partial o_k}{\partial \psi_k} = \sum_{l:(k,l) \in A} \lambda_l w_{lk} r_{lk} \sigma'_k(\psi_k) \quad (13)$$

Clearly if $r_{kj} = 0$ then $\frac{\partial L}{\partial w_{kj}} = 0$ and so w_{kj} is not updated for this mini-batch. At test time, rather than use (10), we use:

$$\psi_k = \sum_{l:(l,k) \in A} w_{kl} p_{kl} o_l + \beta_k \quad (14)$$

4 Batch Normalization

One complication that arises in training deep neural networks is the fact that the distribution of the inputs to each layer change during training as parameters from previous layers are updated. This change is known as internal covariate shift. [IS15] introduced Batch Normalization to normalize layers' inputs during mini-batch training. There is more than one way to normalize; in the discussion below I deviate slightly from [IS15]. Suppose we have a mini-batch $\{x_i, y_i\}_{i=1\dots m}$. During the forward pass, rather than using

$$o_{ki} = \sigma_k(\psi_{ki}) \quad (15)$$

we use

$$o_{ki} = \sigma_k(\hat{\psi}_{ki}) \quad (16)$$

where

$$\hat{\psi}_{ki} = \gamma_k \frac{\psi_{ki} - \hat{\mu}_k}{\sqrt{\hat{\nu}_k^2 + \epsilon}} + \theta_k \quad (17)$$

$$\hat{\mu}_k = \frac{1}{m} \sum_{j=1}^m \psi_{kj} \quad (18)$$

$$\hat{\nu}_k^2 = \frac{1}{m} \sum_{j=1}^m (\psi_{kj} - \hat{\mu}_k)^2 \quad (19)$$

and $\gamma_k, \delta_k \in \mathbb{R}$ are additional parameters to learn. During the backwards pass we treat $\hat{\mu}_k$ and $\hat{\nu}_k^2$ as fixed. Batch Normalization introduces a new term to the equations of back-propagation:

$$\frac{\partial L_i}{\partial \hat{\psi}_{ki}} = \sum_{l:(k,l) \in A} \frac{\partial L_i}{\partial \psi_{li}} \frac{\partial \psi_{li}}{\partial o_{ki}} \frac{\partial o_{ki}}{\partial \hat{\psi}_{ki}} = \sum_{l:(k,l) \in A} \lambda_{li} w_{lk} \sigma'_k(\hat{\psi}_{ki}) \quad (20)$$

so that

$$\lambda_{ki} = \frac{\partial L_i}{\partial \psi_{ki}} = \frac{\partial L_i}{\partial \hat{\psi}_{ki}} \frac{\partial \hat{\psi}_{ki}}{\partial \psi_{ki}} = \sum_{l:(k,l) \in A} \lambda_{li} w_{lk} \sigma'_k(\hat{\psi}_{ki}) \frac{\gamma_k}{\sqrt{\hat{\nu}_k^2 + \epsilon}} \quad (21)$$

The parameter updates for w_{kj} and β_k remain the same as those in (8) and (9) except that now we use (21) to calculate λ_{ki} . Additionally, we need to update the parameters γ_k and θ_k . The chain rule gives us:

$$\frac{\partial L_i}{\partial \gamma_k} = \frac{\partial L_i}{\partial \hat{\psi}_{ki}} \frac{\partial \hat{\psi}_{ki}}{\partial \gamma_k} = \sum_{l:(k,l) \in A} \lambda_{li} w_{lk} \sigma'_k(\hat{\psi}_{ki}) \frac{\psi_{ki} - \hat{\mu}_k}{\sqrt{\hat{\nu}_k^2 + \epsilon}} \quad (22)$$

and

$$\frac{\partial L_i}{\partial \theta_k} = \frac{\partial L_i}{\partial \hat{\psi}_{ki}} \frac{\partial \hat{\psi}_{ki}}{\partial \theta_k} = \sum_{l:(k,l) \in A} \lambda_{li} w_{lk} \sigma'_k(\hat{\psi}_{ki}) \quad (23)$$

so that updates take the form:

$$\gamma_k \leftarrow \gamma_k - \eta \sum_{i=1}^m \sum_{l:(k,l) \in A} \lambda_{li} w_{lk} \sigma'_k \left(\hat{\psi}_{ki} \right) \frac{\psi_{ki} - \hat{\mu}_k}{\sqrt{\hat{\nu}_k^2 + \epsilon}} \quad (24)$$

and

$$\theta_k \leftarrow \theta_k - \eta \sum_{i=1}^m \sum_{l:(k,l) \in A} \lambda_{li} w_{lk} \sigma'_k \left(\hat{\psi}_{ki} \right) \quad (25)$$

5 Layer Normalization

Layer normalization was introduced by [BKH16]

6 Variations of Stochastic Gradient Descent

6.1 Implicit SGD

6.2 Momentum

For the first mini-batch, we use the updates in (3) and (8). This results in a change to the parameters of Δw and $\Delta \beta$. Starting with the second mini-batch, we now follow the updates:

$$\Delta w_{kj} \leftarrow \alpha \Delta w_{kj} - \eta \frac{\partial L}{\partial w_{kj}} \quad \text{and} \quad \Delta \beta_k \leftarrow \alpha \Delta \beta_k - \eta \frac{\partial L}{\partial \beta_k} \quad (26)$$

followed by

$$w \leftarrow w + \Delta w \quad \text{and} \quad \beta \leftarrow \beta + \Delta \beta \quad (27)$$

References

- [BKH16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- [IS15] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.